Functional & Non-Functional Requirements

Real-Time Multi-Threaded Banking Fraud Detection &
Payment Failure Reconciliation System

Theme 5 | Team 11 | Sharangouda • Aditya • Suhas • Rohit | Guide: Dr. Sharada

1. Functional Requirements (FR)

ID	Requirement	Related Foundation Course(s)
FR1	Accept transaction submissions via a REST API (amount, sender, receiver, location, device, merchant details).	Computer Networks
FR2	Validate incoming transaction data before processing.	Software Engineering
FR3	Generate and store a unique idempotency_key per transaction; enforce it with a DB unique constraint to block duplicate retries.	DBMS
FR4	Run 6 independent fraud checks (amount, location, velocity, behavior, merchant, device/IP) concurrently using a shared thread pool.	Operating Systems
FR5	Compute a weighted risk score from the 6 checks and classify each transaction as APPROVE / REVIEW / BLOCK.	DAA
FR6	Apply Z-score anomaly detection on historical numerical behavior (amount, velocity) as a secondary signal.	DAA
FR7	Log each check's score, reason, and executing thread name for auditability.	Operating Systems
FR8	Model each payment as a state machine (INITIATED to COMPLETED, plus TIMEOUT / FAILED / RECONCILIATION_REQUIRED).	DAA, Software Engineering
FR9	Run independent debit-check and credit-check verification threads per transaction.	Operating Systems
FR10	Flag a transaction RECONCILIATION_REQUIRED if a debit is confirmed but no matching credit appears within a timeout.	Operating Systems, Software Engineering
FR11	Record every state transition as an immutable event in an append-only ledger.	DBMS
FR12	For RECONCILIATION_REQUIRED cases, estimate probable issue type, recovery probability, likely recipient, and expected recovery time.	DAA
FR13	Present recovery predictions as estimates only, never as guarantees of refund or settlement.	Software Engineering
FR14	Generate role-specific alerts (sender, receiver, ops) on BLOCK, REVIEW, or RECONCILIATION_REQUIRED outcomes.	OOP (Python)
FR15	Warn a sender not to retry a payment while reconciliation is pending.	Software Engineering

ID	Requirement	Related Foundation Course(s)
FR16	Deliver alerts via dashboard/email.	Computer Networks
FR17	Expose endpoints to query transaction status, risk breakdown, payment state, alerts, cases, and logs.	Computer Networks
FR18	Provide a live dashboard (Streamlit) showing transaction feed, KPI counts, risk breakdowns, reconciliation queue, and logs.	OOP (Python), Software Engineering

2. Non-Functional Requirements (NFR)

ID	Requirement	Related Foundation Course(s)
NFR1	Each transaction's fraud + reconciliation checks shall complete with low latency (synchronous, per-request, not a streaming pipeline).	Operating Systems, DAA
NFR2	Fraud/reconciliation checks shall execute via a shared ThreadPoolExecutor, since checks are I/O-bound, not CPU-bound.	Operating Systems
NFR3	Duplicate prevention shall rely on a database-level unique constraint, not application-level check-then-insert logic.	DBMS, Operating Systems
NFR4	State transitions shall be recorded as an immutable, append-only event log for traceability.	DBMS
NFR5	No real customer PII or live banking data shall be used; all transactions and behavioral data shall be synthetic.	Software Engineering
NFR6	SQLite is acceptable for prototype scale; architecture shall allow migration to PostgreSQL without code changes elsewhere.	DBMS
NFR7	Fraud checks, reconciliation logic, and alerting shall be modular so checks can be added or reweighted independently.	OOP (Python), Software Engineering
NFR8	Dashboard shall present risk scores, payment states, and alerts clearly for non-technical, role-appropriate users.	Software Engineering
NFR9	ML-based recovery estimates shall be clearly distinguished from rule-verified payment states.	Software Engineering, DAA
NFR10	System shall support demonstrable test scenarios (success, failed, pending, timeout, mismatch, duplicate) for grading/viva.	Software Engineering
NFR11	Production-grade security/PCI-DSS is out of scope; basic input validation and idempotency protection are the only guarantees.	Computer Networks, Software Engineering

Note: Course mappings reflect which foundational CS course's concepts the requirement draws on (as identified in the project's Gate 0 presentation), for use in the project report / viva.